

Tree & Graph Traversal Practice Problems

Five warm-up + coding problems focused on in-memory hierarchies, parent chains, dependency ordering, connected components, and subtree aggregation. Each problem ends with a follow-up like the ones asked in interviews.

Problem 1: Directory Paths

Warm-up

You are given an in-memory representation of files and directories. Each node is represented as:

```
class Node:
    id: str
    name: str
    child_ids: list[str]    # empty for files
```

Example (root_id = "r"):

```
r  : name "root",      children [d1, f3]
d1 : name "docs",      children [f1, f2]
f1 : name "resume.pdf", children []
f2 : name "notes.txt",  children []
f3 : name "todo.txt",   children []
```

Given a root_id and a lookup map dict[str, Node], return the maximum depth of the tree, where depth is the number of nodes on the longest path from the root down to any node.

Example output: **3** (root -> docs -> resume.pdf)

Core Problem

Now return the full path of every file in the tree. A path is built by joining the names of every node from the root down to the file, separated by "/" and starting with "/".

Using the same example, the output is:

```
/root/docs/resume.pdf
/root/docs/notes.txt
/root/todo.txt
```

The order of the output does not matter. A node is a file if it has no children.

Follow-up

What happens if the hierarchy is 100,000 levels deep? Besides the recursion limit, think about how much work is spent building path strings at every level. How can you avoid copying the full path at each step?

Problem 2: Company Org Chart

Warm-up

A company stores its org chart as a list of employees. Each employee has:

```
[employee_id, manager_id]
```

The CEO is the only employee whose manager_id is -1. Example:

```
{1, -1}  
{2, 1}  
{3, 1}  
{4, 2}  
{5, 2}  
{6, 3}
```

Write a function that receives an employee ID and returns the list of managers above them, starting with their direct manager and ending with the CEO.

Example:

```
4 -> [2, 1]  
6 -> [3, 1]  
1 -> []
```

Core Problem

Now write a function that takes two employee IDs and returns their **closest common manager**: the lowest person in the org chart who is above both employees (directly or indirectly). If one employee is above the other, that employee is the answer.

Using the same example:

```
(4, 5) -> 2  
(4, 6) -> 1  
(2, 4) -> 2
```

Follow-up

Your function will be called millions of times on the same org chart, and the chart may be 100,000 levels deep. What can you precompute once so that each query is faster than walking both chains in full?

Problem 3: Course Prerequisites

Warm-up

A university lists its courses. Each course has an ID and a list of courses that must be completed before it. Example:

```
CS101   requires nothing
MATH200 requires nothing
CS201   requires CS101
CS301   requires CS201 and MATH200
```

Write a function that returns every course a student can take in their first semester (courses with no prerequisites).

Example output: **CS101, MATH200**

Core Problem

Now return an order in which a student can take every course, so that each course comes after all of its prerequisites. If no such order exists because of circular requirements, return an empty list.

Using the same example, one valid output is:

```
CS101, MATH200, CS201, CS301
```

Any valid order is accepted. Circular example:

```
A requires B
B requires A
Output: []
```

Every prerequisite listed is guaranteed to be a course in the list.

Follow-up

A student can take any number of courses in one semester, as long as all prerequisites were finished in earlier semesters. What is the minimum number of semesters needed to finish every course? (For the example above, the answer is 3.)

Problem 4: Server Clusters

Warm-up

A data center has servers numbered 1 to n . Some pairs of servers are connected by a network cable. Cables work in both directions, and a message can travel through any number of cables. Example:

```
n = 6
cables = {
  {1, 2},
  {2, 3},
  {4, 5}
}
```

Write a function that takes two servers and returns true if a message can get from one to the other. Otherwise, return false.

```
(1, 3) -> true
(1, 4) -> false
```

Core Problem

A **cluster** is a group of servers that can all reach each other. A server with no cables is a cluster by itself.

Return two values: the number of clusters, and the number of servers in the largest cluster.

Using the same example, the clusters are {1, 2, 3}, {4, 5}, and {6}. Therefore the answer is:

```
clusters = 3, largest = 3
```

The cable list may contain cycles (for example 1-2, 2-3, 3-1). Your solution must still terminate.

Follow-up

Now cables are installed one at a time, and after each new cable you must report the current number of clusters. Re-running a full traversal after every cable is too slow for 1,000,000 cables. What data structure lets you answer each update in nearly constant time?

Problem 5: Disk Cleanup

Warm-up

You are given an in-memory representation of files and directories. Each node is represented as:

```
class Node:
    id: str
    size: int          # 0 for directories, positive for files
    child_ids: list[str]
```

Example (root_id = "root"):

```
root : size 0,    children [a, b, f1]
a    : size 0,    children [f2, f3]
b    : size 0,    children [c, f5]
c    : size 0,    children [f4]
f1   : size 100
f2   : size 300
f3   : size 200
f4   : size 420
f5   : size 50
```

Given a root_id and a lookup map dict[str, Node], return the ID of the largest single file.

Example output: **f4**

Core Problem

The disk is almost full, and you need to free at least X bytes by deleting exactly one directory (which deletes everything inside it). The root counts as a directory.

Return the ID of the **smallest** directory whose total size is at least X. If no directory is large enough, return an empty string.

Directory sizes in the example: a = 500, c = 420, b = 470, root = 1070.

```
X = 400  -> c      (420)
X = 450  -> b      (470)
X = 2000 -> ""
```

Your solution should compute every directory's total size without re-traversing the same subtree more than once.

Follow-up

File sizes now change constantly (logs growing, files being overwritten), and you need every directory's total size to stay correct after each change. How do you update the totals without recomputing the whole tree?